Vídeo 5.4: Arrumando a casa (organizando o código em camadas)

Duração-alvo: 7 min | **Conceito:** separação em camadas | **O aluno sai sabendo:** por que dividir o código em partes e o que cada parte faz.

Antes de gravar: partir do código do vídeo 5.3. Este vídeo mexe em muitos arquivos, então o ideal é deixar a estrutura pronta e gravar mostrando cada arquivo e explicando a sua função, com cortes na edição, em vez de digitar tudo ao vivo. A API deve responder exatamente igual no fim. Ao fim da gravação, marcar o código como versão v5.

[0:00] Abertura

Na tela: título do vídeo e o server.js longo, rolando.

Fala: "A nossa API funciona, valida os dados e trata erros. Mas olhe o arquivo server ponto js: ele já tem muita coisa, e vai crescer. Neste vídeo vamos arrumar a casa: separar o código em partes, cada uma com uma tarefa. A API vai responder exatamente igual. Só o código fica mais fácil de ler, de mudar e, no próximo módulo, de testar."

[0:45] A biblioteca organizada

Na tela: ilustração com três áreas: balcão, depósito e regulamento.

Fala: "Pense na biblioteca: o balcão atende, o depósito guarda, o regulamento diz o que é permitido. Cada um tem a sua função. Se o balcão também cuidasse do depósito e escrevesse o regulamento, ficaria tudo confuso. Em código é igual. Vamos separar em camadas: as rotas, que recebem o pedido e devolvem a resposta; a validação, que confere os dados; e o acesso ao banco, que guarda e busca."

[1:30] A nova estrutura

Na tela: a árvore de arquivos abaixo, com os arquivos aparecendo um de cada vez.

```
src/
  server.js              liga o servidor
  app.js                 monta o app: middlewares e rotas
  db.js                  abre o banco
  repositorio/livros.js  comandos SQL dos livros
  middlewares/validacao.js  validarLivro
  rotas/livros.js        rotas de /livros
```

Fala: "Esta será a nova estrutura, dentro de uma pasta chamada src. O server ponto js só liga o servidor. O app ponto js monta o app. O db ponto js abre o banco. O repositório guarda os comandos SQL. Os middlewares guardam a validação. E rotas guarda as rotas dos livros. Vamos ver o papel de cada arquivo."

[2:15] O banco e o repositório

Na tela: mostrar src/db.js e src/repositorio/livros.js, com o cursor passando pelas partes explicadas.

```
// src/db.js
 const sqlite3 = require('sqlite3');
 const { open } = require('sqlite');

 let conexao;

 async function obterBanco() {
   if (!conexao) {
          conexao = await open({
          filename: process.env.ARQUIVO_BANCO || 'biblioteca.db',
          driver: sqlite3.Database
          });

          await conexao.exec(`
       CREATE TABLE IF NOT EXISTS livros (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          titulo TEXT NOT NULL,
          autor TEXT NOT NULL
          )
          `);
   }

   return conexao;
 }

 module.exports = { obterBanco };


// src/repositorio/livros.js
 const { obterBanco } = require('../db');

 async function listar(autor) {
   const db = await obterBanco();

   if (autor) {
          return db.all('SELECT * FROM livros WHERE autor LIKE ?', `%${autor}%`);
   }

   return db.all('SELECT * FROM livros');
 }

 async function buscarPorId(id) {
   const db = await obterBanco();
   return db.get('SELECT * FROM livros WHERE id = ?', id);
 }

 async function criar(titulo, autor) {
   const db = await obterBanco();
```

```
  const resultado = await db.run(
        'INSERT INTO livros (titulo, autor) VALUES (?, ?)',
        titulo, autor
  );
  return { id: resultado.lastID, titulo, autor };
}

async function atualizar(id, titulo, autor) {
  const db = await obterBanco();
  const resultado = await db.run(
        'UPDATE livros SET titulo = ?, autor = ? WHERE id = ?',
        titulo, autor, id
  );

  if (resultado.changes === 0) {
        return null;
  }

  return { id: Number(id), titulo, autor };
}

async function excluir(id) {
  const db = await obterBanco();
  const resultado = await db.run('DELETE FROM livros WHERE id = ?', id);
  return resultado.changes > 0;
}

module.exports = { listar, buscarPorId, criar, atualizar, excluir };
```

Fala: “Primeiro o db ponto js. É a função abrirBanco de antes, com uma diferença: agora ela se chama obterBanco, e só abre o banco na primeira vez. Nas próximas, devolve a conexão que já existe. O nome do arquivo do banco pode vir de uma configuração de fora e, se não vier, usa biblioteca ponto db. Vamos precisar disso no módulo de testes. Agora o repositório. Ele é o único lugar que conhece o SQL. Cada função faz uma coisa: listar, buscar, criar, atualizar, excluir. As rotas passam a chamar essas funções, sem saber como o banco funciona por dentro. Se um dia trocarmos de banco, só este arquivo muda. Repare que atualizar devolve null quando não encontra o livro, e excluir devolve verdadeiro ou falso, para a rota decidir a resposta.”

[3:45] A validação e as rotas

Na tela: mostrar src/middlewares/validacao.js, que contém a função validarLivro do vídeo anterior, e depois src/rotas/livros.js.

```
// src/middlewares/validacao.js
function validarLivro(req, res, next) {
  const { titulo, autor } = req.body ?? {};
  const erros = [];

  if (typeof titulo !== 'string' || titulo.trim() === '') {
```

```
        erros.push('O título é obrigatório e deve ser um texto.');
  }

  if (typeof autor !== 'string' || autor.trim() === '') {
        erros.push('O autor é obrigatório e deve ser um texto.');
  }

  if (erros.length > 0) {
        return res.status(400).json({ erro: 'Dados inválidos', detalhes: erros });
  }

  next();
}

module.exports = { validarLivro };


// src/rotas/livros.js
const express = require('express');
const repositorio = require('../repositorio/livros');
const { validarLivro } = require('../middlewares/validacao');

const router = express.Router();

router.get('/', async (req, res) => {
  const livros = await repositorio.listar(req.query.autor);
  res.json(livros);
});

router.get('/:id', async (req, res) => {
  const livro = await repositorio.buscarPorId(req.params.id);

  if (!livro) {
        return res.status(404).json({ erro: 'Livro não encontrado' });
  }

  res.json(livro);
});

router.post('/', validarLivro, async (req, res) => {
  const { titulo, autor } = req.body;
  const novo = await repositorio.criar(titulo, autor);
  res.status(201).json(novo);
});

router.put('/:id', validarLivro, async (req, res) => {
  const { titulo, autor } = req.body;
  const livro = await repositorio.atualizar(req.params.id, titulo, autor);
```

```
  if (!livro) {
        return res.status(404).json({ erro: 'Livro não encontrado' });
  }

  res.json(livro);
});

router.delete('/:id', async (req, res) => {
  const excluiu = await repositorio.excluir(req.params.id);

  if (!excluiu) {
        return res.status(404).json({ erro: 'Livro não encontrado' });
  }

  res.status(204).send();
});

module.exports = router;
```

Fala: “A validação é a mesma função, só mudou de arquivo. As rotas ficaram bem mais curtas. Cada uma pega os dados do pedido, chama o repositório e escolhe a resposta. O Router é um mini-app só para as rotas de livros. Dentro dele, os endereços são relativos: a barra sozinha vale para barra livros, e barra dois pontos id vale para barra livros barra id. O prefixo barra livros será definido uma vez só, no app ponto js.”

[5:15] O app e o servidor

Na tela: mostrar src/app.js e src/server.js.

```
// src/app.js
const express = require('express');
const livrosRotas = require('./rotas/livros');

const app = express();

app.use(express.json());

app.use((req, res, next) => {
  console.log(`${req.method} ${req.originalUrl}`);
  next();
});

app.get('/status', (req, res) => {
  res.json({ ok: true, mensagem: 'API da biblioteca funcionando' });
});

app.use('/livros', livrosRotas);

app.use((req, res) => {
```

```
  res.status(404).json({ erro: 'Rota não encontrada' });
});

app.use((erro, req, res, next) => {
  console.error(erro);
  res.status(500).json({ erro: 'Erro interno do servidor' });
});

module.exports = app;


// src/server.js
const app = require('./app');

const porta = 3000;

app.listen(porta, () => {
  console.log(`Servidor rodando em http://localhost:${porta}`);
});
```

Fala: “O app ponto js monta tudo: lê JSON, anota os pedidos, tem a rota de status, entrega tudo que começa com barra livros às rotas de livros e termina com os dois tratadores que escrevemos. E, no fim, exporta o app. O server ponto js só importa o app e liga na porta. Por que separar? Porque, no próximo módulo, as ferramentas de teste vão usar o app sem ligar um servidor de verdade.”

[5:55] Testando

Na tela: no terminal, rodar node src/server.js, executar um passeio completo de CRUD e um pedido inválido no requisicoes.http, e mostrar que os livros que já estavam no biblioteca.db continuam lá.

Fala: “Rodo o servidor, agora com node, espaço, src barra server ponto js. Faço um passeio completo: cadastrar, listar, alterar, excluir e um pedido inválido. Tudo responde igual. E repare: os livros que já estavam guardados continuam lá, porque o banco é o mesmo arquivo. Reorganizamos o código sem mudar o comportamento. Esse é o objetivo de arrumar a casa.”

[6:40] Resumo

Na tela: a árvore de arquivos com uma frase ao lado de cada pasta.

Fala: “Resumindo: separamos o código em camadas. As rotas cuidam de pedido e resposta, o middleware cuida da validação, o repositório cuida do banco, e o app monta tudo. Cada parte tem uma tarefa só. No próximo módulo, vamos testar a API e documentá-la.”

Exercício sugerido: criar uma nova rota GET /livros/total que devolva quantos livros existem, decidindo em qual arquivo cada pedaço do código deve ficar (dica: a ordem das rotas importa, então a nova rota precisa vir antes da rota que usa o parâmetro id).